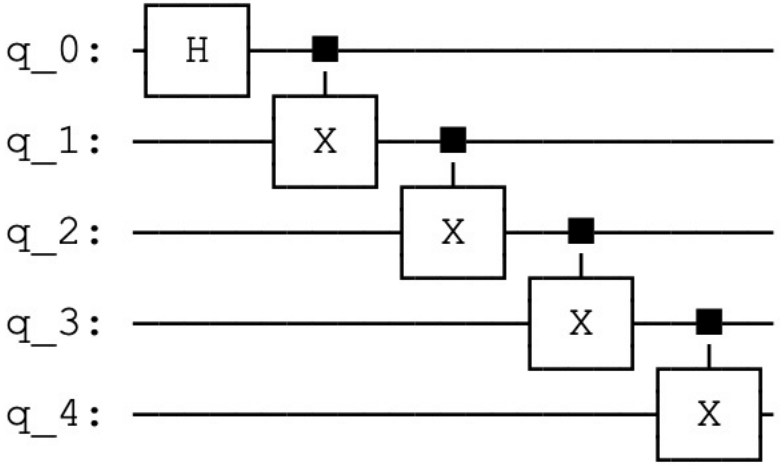
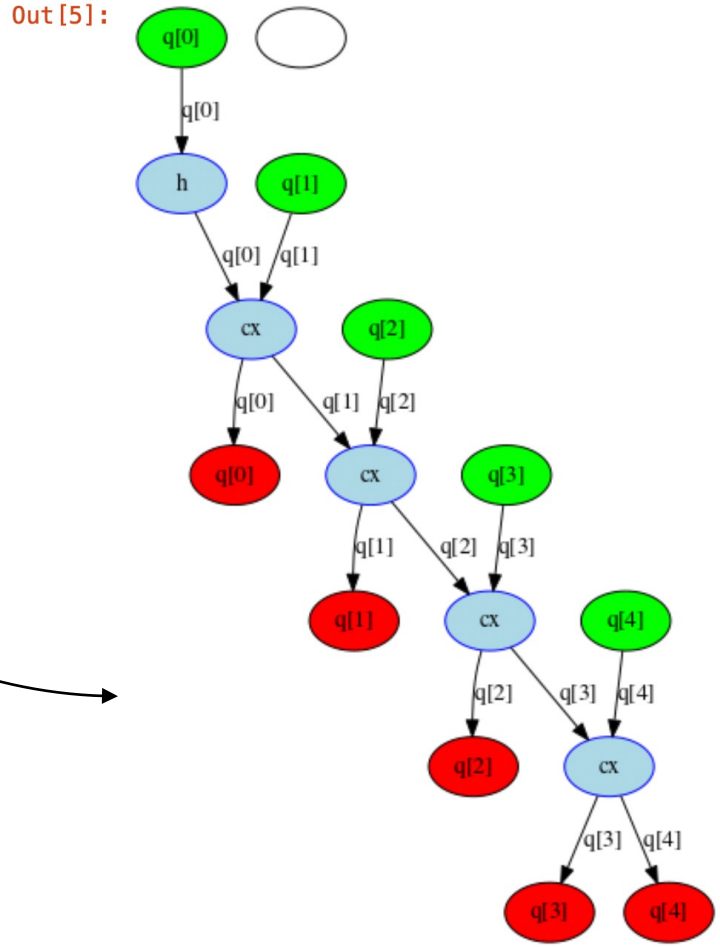
What are Tensor Networks (TN)?



Most quantum circuits can be represented as directed graphs



Quantum Simulation using TN (reference)

1. Build the tensor network representation of the circuit,
2. Perform local simplifications on the tensor network to make it easier to contract.
3. Find a contraction path for this simplified tensor network. This a series of pairwise tensor contractions that turn the tensor network into a single tensor - represented by a binary contraction tree.
4. Optionally slice (or 'chunk') the contraction, breaking it into many independent, smaller contractions, either to fit memory constraints or introduce embarrassing parallelism.
5. Perform the contraction! (numpy/cupy/jax/torch etc)

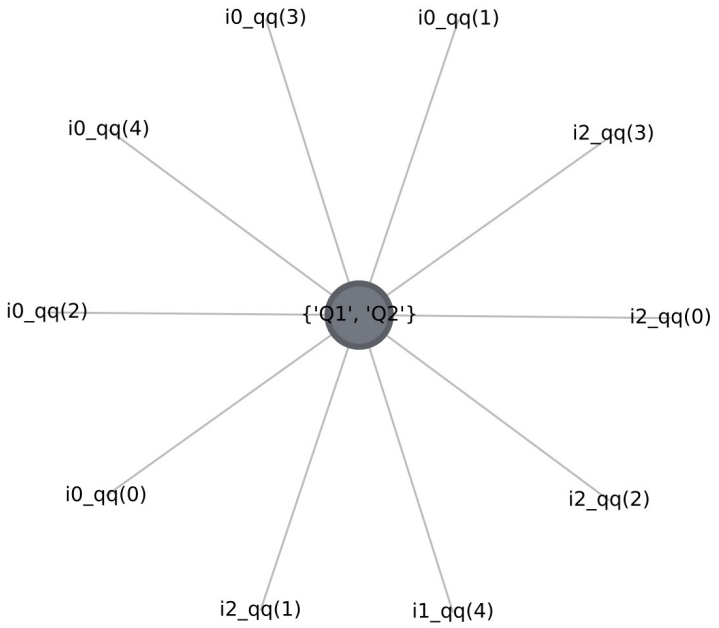
   **** We want to create a new library that is specialized of quantum simulations**

What happens in step 4

The optimal contraction path is identified, which looks like this for the example in slide-1.

```
eq - ka,blkc,dmle,fnmg,hinj->abcdefghij
path -  ((0, 1), (0, 1), (0, 2), (0, 1))
expression -  <ContractExpression('ka,blkc,dmle,fnmg,hinj->abcdefghij')>
  1.  'blkc,ka->blca' [TDOT]
  2.  'fnmg,dmle->fngdle' [TDOT]
  3.  'fngdle,hinj->fgdlehij' [TDOT]
  4.  'fgdlehij,blca->abcdefghij' [TDOT]
```

And then "contraction" (gate multiplication) is performed, shrinking our graph to just one node like so-



Final Unitary representation of circuit

- Final unitary (32 x 32) has-
  - 93.75% overall sparsity (64 out of 1024)

```
Out[14]: [[ 0.707107+0.j  0.      +0.j  0.707107+0.j  0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j
            0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j
            0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j
            0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j]
          [ 0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j  0.707107+0.j  0.      +0.j  0.707107+0.j
            0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j
            0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j
            0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j]
          [ 0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j
            0.707107+0.j  0.      +0.j -0.707107+0.j  0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j
            0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j
            0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j]
          [ 0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j
            0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j  0.707107+0.j  0.      +0.j -0.707107+0.j
            0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j
            0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j]
          [ 0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j  0.707107+0.j  0.      +0.j  0.707107+0.j  0.      +0.j
            0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j  0.      +0.j
```

What's cool about this? **After constants are factored out, the unitaries mostly only contain 0, 1, -1**.

One thought is, we can speedup operations between these by not performing arithmetic operations at all and just sticking to comparisons. For example- if a == 1 and b == 1, return 2

Problem Statement (subject to change)

- Analyze intermediate unitaries for sparsity.
  - Follow the contraction path (opt_einsum) and analyze how the unitary of the circuit evolves over time steps.
- Write a custom numerical library that takes quantum sparsity patterns into account (diagonal/anti-diagonal, (1/-1/0) etc)
  - New data structure format to store sparse unitaries efficiently.
  - Conversions between this new format and other formats.
  - Kernels:
    - Vector to Vector multiplication
    - Vector to Matrix multiplication
    - Matrix to Matrix multiplication
  - As a fall back, leverage NumPy for dense representations and kernels.